



使用RandomAccessFile

北京理工大学计算机学院
金旭亮

应用场景



问：

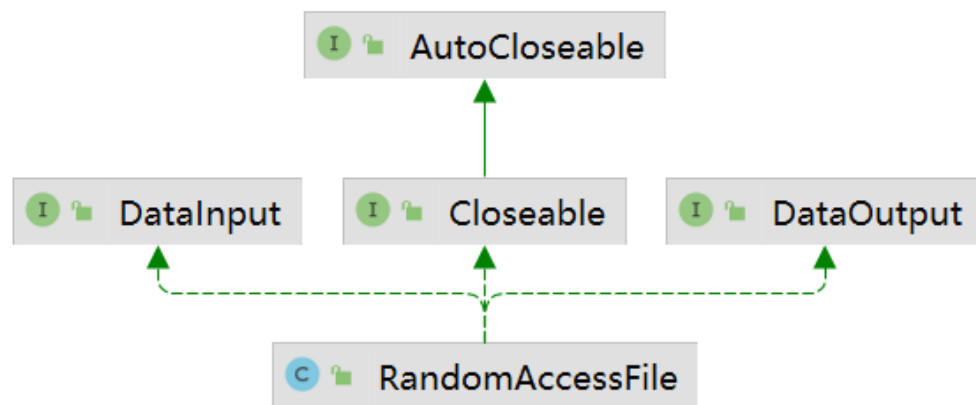
通常情况下，我们是以“要不读”或“要不写”两种模式来存取文件的，但如果需要“又读又写”同一个文件，怎么办？

答：

可以使用RandomAccessFile实现目的。

RandomAccessFile的相关类型

I DataInput		
(m)	readBoolean()	boolean
(m)	readByte()	byte
(m)	readChar()	char
(m)	readDouble()	double
(m)	readFloat()	float
(m)	readFully(byte[])	void
(m)	readFully(byte[], int, int)	void
(m)	readInt()	int
(m)	readLine()	String
(m)	readLong()	long
(m)	readShort()	short
(m)	readUTF()	String
(m)	readUnsignedByte()	int
(m)	readUnsignedShort()	int
(m)	skipBytes(int)	int



JDK中的RandomAccessFile实现了DataInput和DataOutput两个接口所定义的读写方法。

I DataOutput		
(m)	write(byte[])	void
(m)	write(byte[], int, int)	void
(m)	write(int)	void
(m)	writeBoolean(boolean)	void
(m)	writeByte(int)	void
(m)	writeBytes(String)	void
(m)	writeChar(int)	void
(m)	writeChars(String)	void
(m)	writeDouble(double)	void
(m)	writeFloat(float)	void
(m)	writeInt(int)	void
(m)	writeLong(long)	void
(m)	writeShort(int)	void
(m)	writeUTF(String)	void

构造RandomAccessFile对象



```
var rf = new RandomAccessFile("文件名", "打开模式");  
//使用rf进行读写...
```

可用的文件打开模式:

文件打开模式	说明
r	只读
rw	读写。如果文件不存在，会创建它。
rws	功能同rw，只不过在更新文件内容时，文件的元数据（metadata）也一并更新到磁盘文件中
rwd	功能同rw，对文件内容的更新，会被立即写入到磁盘文件。

关于随机随取的文件-1



支持随机存取的文件拥有一个“**文件指针 (file pointer)**”，刚打开文件时，这个指针指向位置0，随着不断地向文件中读或写数据，这些指针随之移动。如果尝试写入的位置比文件当前长度更长，则文件会自动扩展。



调用RandomAccessFile的getFilePointer()方法可以获取当前文件指针的位置。



可以使用seek()方法移动文件指针到特定的位置。

关于随机随取的文件-2



可以通过`setLength()`方法显式设置文件长度，从而动态地扩充或截断文件。



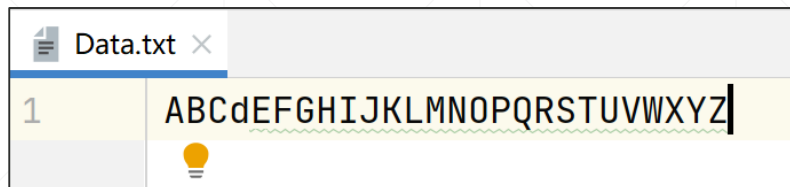
当操作系统打开一个文件时，它会创建一个数据结构，称为"**文件句柄 (file handle)**"，来代表这个文件。在Java中，使用`java.io.FileDescriptor`类封装这个数据结构相关的信息。



`FileDescriptor`类定义了一个`sync()`方法通知操作系统将保存于“缓冲区”的数据写入到真实的文件中。

通过示例学用RandomAccessFile

在一个数据文件中，保存有英文的26个字母



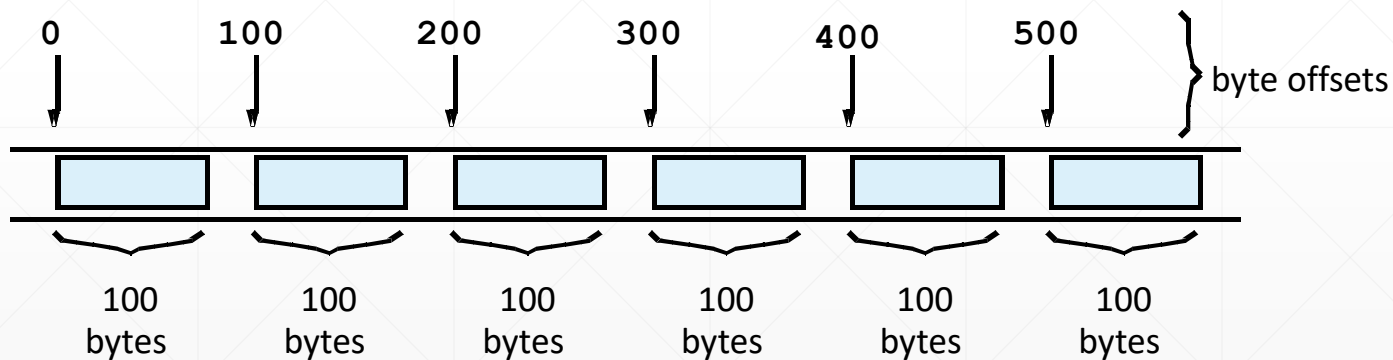
以下示例，展示了它的基本用法：

```
public class UseRandomAccessFileDemo {  
    public static void main(String[] args) throws IOException {  
        var rf = new RandomAccessFile(name: "Data.txt", mode: "rw");  
        System.out.println((char) rf.read()); //A  
        System.out.println((char) rf.read()); //B  
        System.out.println((char) rf.read()); //C  
        rf.write(b: 'd'); //写入d  
        System.out.println((char) rf.read()); //E  
        rf.skipBytes(n: 3); //跳过3个字节  
        System.out.println((char) rf.read()); //I  
        rf.seek(pos: 3); //回到索引值为3的位置  
        System.out.println((char) rf.read()); //d
```

```
        System.out.println((char) rf.read()); //d  
        //获取文件当前读写指针，即当前读写的位置索引  
        System.out.println(rf.getFilePointer()); //4  
        //向后移动两个字节  
        rf.seek(pos: rf.getFilePointer() + 2);  
        System.out.println((char) rf.read()); //G  
    }  
}
```

应用实例：实现一个简单的文件型数据库

如果能确保每条记录都是定长的，那么，使用RandomAccessFile可以很方便地实现一个“文件型数据库”，它提供了诸如seek, readXXX, writeXXX之类的方法，可以很方便地操作单条记录。



示例运行截图



```
Run: StudentFileDB x
"C:\Program Files\Java\jdk-17\bin\java.exe"
读取第几笔数据? 2
姓名: 李四
分数: 95
Process finished with exit code 0
```

- 如果选择的记录数太大, 引发java.io.EOFException

```
读取第几笔数据? 200
java.io.EOFException Create breakpoint
    at java.base/java.io.RandomAccessFile.readChar(RandomAccessFile.java:816)
    at RandomAccessFileDemo.readName(RandomAccessFileDemo.java:56)
    at RandomAccessFileDemo.main(RandomAccessFileDemo.java:36)
```

RandomAccessFileDemo.java

示例分析

c Student	
m	Student()
m	Student(String, int)
m	size() int
p	name String
p	score int

文件中每条记录的格式

最长15个字符

int 类型，4个字节



```
public class StudentFileDB {  
    public static void main(String[] args) {  
        //测试用示例数据  
        Student[] students = {  
            new Student( name: "张三", score: 90),  
            new Student( name: "李四", score: 95),  
            new Student( name: "王五", score: 88),  
            new Student( name: "赵六", score: 84)};}
```

记录的写入

记录格式

学生姓名	考试成绩
------	------

```
File file = new File("student.dat");  
// 建立RandomAccessFile实例并以读写模式开启文件  
// 利用try with resource特性, 在用完后自动关闭文件  
try (var randomAccessFile = new RandomAccessFile(file, "rw")) {  
  
    for (Student value : students) {  
        // 使用对应的write方法写入数据  
        randomAccessFile.writeChars(value.getName());  
        randomAccessFile.writeInt(value.getScore());  
    }  
}
```



出于简单起见, 这里没有考虑学生姓名很长的场景。

读取学生姓名

//读取学生姓名的辅助方法

1 usage

```
private static String readName(  
    RandomAccessFile randomAccessfile)  
    throws IOException {  
    char[] name = new char[15];  
    for (int i = 0; i < name.length; i++)  
        name[i] = randomAccessfile.readChar();  
    // 将空字符取代为空格符并返回  
    return new String(name).replace( oldChar: '\\0', newChar: ' ');  
}
```

按位置查询记录

```
Scanner scanner = new Scanner(System.in);  
System.out.print("读取第几笔数据? ");  
int num = scanner.nextInt();
```

// 使用seek()方法操作存取位置

```
randomAccessFile.seek( pos: (long) (num - 1) * Student.size());  
Student student = new Student();
```

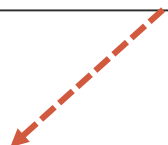
// 使用对应的read方法读出数据

```
student.setName(readName(randomAccessFile));  
student.setScore(randomAccessFile.readInt());
```

```
System.out.println("姓名: " + student.getName());  
System.out.println("分数: " + student.getScore());
```

// 每笔数据固定写入34字节

```
public static int size() {  
    return 34;  
}
```



进一步完善.....



StudentFileDB这个示例，仅实现了最基础的“添加记录”、“按指定位置读取记录”的功能，并且没有对数据的有效性进行任何的检测工作。



请在本讲示例的基础之上，将此应用完善为一个功能完备的，支持CRUD（增、删、改、查）功能的文件型数据库应用。

进一步完善.....



如果是变长记录，可以在每条记录前固定4个字节用于保存本记录长度，当需要读取第 n 条记录时，从前到后顺序扫描，即可计算出位移数值。



如果变长记录并且限定了最大记录数，则可以在文件开头创建一个固定大小的数据区，在这里存储文件中当前已经存放的记录信息，从而实现快速的存取。

同学们可以依据以上构想，编写相应的示例程序进行实践。

实战课题



随机存取文件，最适合于文件记录固定长度的场景，因为可以很容易地计算出特定记录距文件开头的位移。



如果以单个字节为存取单元，则可以使用它来开发一个二进制**文件编辑器**，因为可以移动文件读写指针到任意一个位置，然后读入或写出任意多个字节到文件中。

请你一试身手！

实战课题



随机存取的文件，还可以用于开发多线程文件下载器。



当从互联网上下载比较大的文件时，可以启动多个线程，每个线程下载文件的一部分，然后，将其写入到本地磁盘文件中。



比较著名的工具软件有“迅雷”下载。



你能否自己写出一个简易版的“迅雷”出来？